状态与副作用

回顾fiber 数据结构, 并结合前文 `fiber树构造` 系列的解读, 我们注意到 `fiber` 众多属性中, 有 2 类属性十分关键:

1. `fiber` 节点的自身状态: 在 `renderRootSync[Concurrent]` 阶段, 为子节点提供确定的输入数据, 直接影响子节点的生成.
2. `fiber` 节点的副作用: 在 `commitRoot` 阶段, 如果 `fiber` 被标记有副作用, 则副作用相关函数会被(同步/异步)调用.

```
export type Fiber = {
  // 1. fiber节点自身状态相关
  pendingProps: any,
  memoizedProps: any,
  updateQueue: mixed,
  memoizedState: any,
  // 2. fiber节点副作用(Effect)相关
  flags: Flags,
  subtreeFlags: Flags, // v17.0.2未启用
  deletions: Array<Fiber> | null, // v17.0.2未启用
  nextEffect: Fiber | null,
  firstEffect: Fiber | null,
  lastEffect: Fiber | null,
};
```

状态

与 `状态` 相关有 4 个属性:

1. `fiber.pendingProps` : 输入属性, 从 `ReactElement` 对象传入的 props. 它和 `fiber.memoizedProps` 比较可以得出属性是否变动.
2. `fiber.memoizedProps` : 上一次生成子节点时用到的属性, 生成子节点之后保持在内存中. 向下生成子节点之前叫做 `pendingProps` , 生成子节点之后会把 `pendingProps` 赋值给 `memoizedProps` 用于下一次比较. `pendingProps` 和 `memoizedProps` 比较可以得出属性是否变动.
3. `fiber.updateQueue` : 存储 `update更新对象` 的队列, 每一次发起更新, 都需要在该队列上创建一个 `update对象` .

4. `fiber.memoizedState`: 上一次生成子节点之后保持在内存中的局部状态.

它们的作用只局限于 `fiber树构造` 阶段, 直接影响子节点的生成.

副作用

与 `副作用` 相关有 4 个属性:

1. `fiber.flags`: 标志位, 表明该 `fiber` 节点有副作用.
2. `fiber.nextEffect`: 单向链表, 指向下一个副作用 `fiber` 节点.
3. `fiber.firstEffect`: 单向链表, 指向第一个副作用 `fiber` 节点.
4. `fiber.lastEffect`: 单向链表, 指向最后一个副作用 `fiber` 节点.

通过前文 `fiber树构造` 我们知道, 单个 `fiber` 节点的副作用队列最后都会上移到根节点上. 所以在 `commitRoot` 阶段中, `react` 提供了 3 种处理副作用的方式

另外, `副作用` 的设计可以理解为对 `状态` 功能不足的补充.

- `状态` 是一个 `静态` 的功能, 它只能为子节点提供数据源.
- 而 `副作用` 是一个 `动态` 功能, 由于它的调用时机是在 `fiber树渲染阶段`, 故它拥有更多的能力, 能轻松获取 `突变前快照， 突变后的DOM节点等`. 甚至通过 `调用api` 发起新的一轮 `fiber树构造`, 进而改变更多的 `状态`, 引发更多的 `副作用`.

外部 api

`fiber` 对象的这 2 类属性, 可以影响到渲染结果, 但是 `fiber` 结构始终是一个内核中的结构, 对于外部来讲是无感知的, 对于调用方来讲, 甚至都无需知道 `fiber` 结构的存在. 所以正常只有通过暴露 `api` 来直接或间接的修改这 2 类属性.

class 组件

```
class App extends React.Component {
  constructor() {
    this.state = {
      // 初始状态
      a: 1,
    };
  }
  changeState = () => {
    this.setState({ a: ++this.state.a }); // 进入reconciler流程
  };
  // 生命周期函数: 状态相关
```

```
  static getDerivedStateFromProps(nextProps, prevState) {
    console.log('getDerivedStateFromProps');
    return prevState;
  }
  // 生命周期函数: 状态相关
  shouldComponentUpdate(newProps, newState, nextContext) {
    console.log('shouldComponentUpdate');
    return true;
  }
  // 生命周期函数: 副作用相关 fiber.flags |= Update
  componentDidMount() {
    console.log('componentDidMount');
  }
  // 生命周期函数: 副作用相关 fiber.flags |= Snapshot
  getSnapshotBeforeUpdate(prevProps, prevState) {
    console.log('getSnapshotBeforeUpdate');
  }
  // 生命周期函数: 副作用相关 fiber.flags |= Update
  componentDidUpdate() {
    console.log('componentDidUpdate');
  }
  render() {
    // 返回下级ReactElement对象
    return <button onClick={this.changeState}>{this.state.a}</button>;
  }
}
```

1. 状态相关: `fiber树构造` 阶段.
   a. 构造函数: `constructor` 实例化时执行, 可以设置初始 state, 只执行一次.
   b. 生命周期: `getDerivedStateFromProps` 在 `fiber树构造` 阶段 (`renderRootSync[Concurrent]`)执行, 可以修改 state(链接).
   c. 生命周期: `shouldComponentUpdate` 在, `fiber树构造` 阶段 (`renderRootSync[Concurrent]`)执行, 返回值决定是否执行 render(链接).
2. 副作用相关: `fiber树渲染` 阶段.
   a. 生命周期: `getSnapshotBeforeUpdate` 在 `fiber树渲染` 阶段 (`commitRoot>commitBeforeMutationEffects>commitBeforeMutationEffectOnFiber`)执行(链接).
   b. 生命周期: `componentDidMount` 在 `fiber树渲染` 阶段 (`commitRoot>commitLayoutEffects->commitLayoutEffectOnFiber`)执行(链接).
   c. 生命周期: `componentDidUpdate` 在 `fiber树渲染` 阶段 (`commitRoot>commitLayoutEffects->commitLayoutEffectOnFiber`)执行(链

接).

可以看到, 官方 `api` 提供的 `class组件` 生命周期函数实际上也是围绕 `fiber树构造` 和 `fiber树渲染` 来提供的.

function 组件

注: `function组件` 与 `class组件` 最大的不同是: `class组件` 会实例化一个 `instance` 所以拥有独立的局部状态; 而 `function组件` 不会实例化, 它只是被直接调用, 故无法维护一份独立的局部状态, 只能依靠 `Hook` 对象间接实现局部状态(有关更多 `Hook` 实现细节, 在 `Hook原理` 章节中详细讨论).

在 `react` 中共定义了14 种 Hook, 其中最常用的 `useState, useEffect, useLayoutEffect 等`

```
function App() {
  // 状态相关: 初始状态
  const [a, setA] = useState(1);
  const changeState = () => {
    setA(++a); // 进入reconciler流程
  };
  // 副作用相关: fiber.flags |= Update | Passive;
  useEffect(() => {
    console.log(`useEffect`);
  }, []);
  // 副作用相关: fiber.flags |= Update;
  useLayoutEffect(() => {
    console.log(`useLayoutEffect`);
  }, []);
  // 返回下级ReactElement对象
  return <button onClick={changeState}>{a}</button>;
}
```

1. 状态相关: `fiber树构造` 阶段.
   a. `useState` 在 `fiber树构造` 阶段(`renderRootSync[Concurrent]`)执行, 可以修改 `Hook.memoizedState` .
2. 副作用相关: `fiber树渲染` 阶段.
   a. `useEffect` 在 `fiber树渲染` 阶段(`commitRoot->commitBeforeMutationEffects->commitBeforeMutationEffectOnFiber`)执行(注意是异步执行, 链接).

b. `useLayoutEffect` 在 `fiber树渲染` 阶段(`commitRoot->commitLayoutEffects->commitLayoutEffectOnFiber->commitHookEffectListMount`)执行(同步执行, 链接).

细节与误区

这里有 2 个细节:

1. `useEffect(function(){}, [])` 中的函数是异步执行, 因为它经过了调度中心
2. `useLayoutEffect` 和 `Class组件` 中的 `componentDidMount,componentDidUpdate` 从调用时机上来讲是等价的, 因为他们都在 `commitRoot->commitLayoutEffects` 函数中被调用。误区: 虽然官网文档推荐尽可能使用标准的 `useEffect` 以避免阻塞视觉更新 , 所以很多开发者使用 `useEffect` 来代替 `componentDidMount,componentDidUpdate` 是不准确的, 如果完全类比, `useLayoutEffect` 比 `useEffect` 更符合 `componentDidMount,componentDidUpdate` 的定义.